
CPU Evaluation of KV Cache Compression Methods on Pythia

Anonymous Author(s)

Affiliation

Address

email

1 Implementation

The benchmark is implemented in `pythia_kvpress_test.py` and was run as:

```
python pythia_kvpress_test.py --device cpu
```

It evaluates EleutherAI/pythia-70m on CPU in float32. The dense baseline, NoPress, leaves the KV cache unchanged. The reproduced methods are KnormPress, which scores tokens from key norms; SnapKVPress, which scores old tokens by recent-window attention; AdaKVPress, which applies adaptive head-wise pruning on top of SnapKV scores; and StreamingLLMPress, which keeps sink tokens and recent tokens while pruning the middle context.

Perplexity is computed after a 1024-token prefill and on 256 target tokens. For compressed methods, the prefill runs inside the press hook, so target scoring uses the compressed cache. Decode throughput is measured only after prefill and compression; for none, timing starts after prefill only. Therefore the reported throughput isolates the benefit of a smaller KV cache during decoding, not the one-time compression cost.

2 Experimental Setup

Table 1 summarizes the reproduced CPU run. WikiText-2 uses three samples. PG-19 uses one local sample from `data/pg19_sample.txt`, so its numbers are less stable than the WikiText-2 results.

Submitted to 39th Conference on Neural Information Processing Systems (NeurIPS 2025). Do not distribute.

Table 1: Benchmark configuration for `pythia_kvpress_test_results.csv`.

Parameter	Value
Model	EleutherAI/pythia-70m
Device / dtype	CPU / float32
Datasets	WikiText-2, PG-19
Context tokens	1024
Target tokens for PPL	256
Generated tokens for throughput	64
Compression ratios	0.2, 0.4, 0.5, 0.6, 0.8
Warmup windows	3
Throughput repetitions	3
SnapKV window size	64

Table 2: WikiText-2 CPU results. PPL is lower better; throughput is higher better.

Method	0.2		0.4		0.5		0.6		0.8	
	PPL	tok/s	PPL	tok/s	PPL	tok/s	PPL	tok/s	PPL	tok/s
none	32.30	65.89	32.30	65.89	32.30	65.89	32.30	65.89	32.30	65.89
adakv	44.55	72.27	49.80	80.31	50.95	76.64	53.17	66.54	59.90	77.75
knorm	35.43	72.18	42.94	91.44	46.69	92.96	48.06	94.05	56.43	99.22
snapkv	33.65	92.67	35.89	84.65	37.50	81.69	39.90	95.28	43.68	102.23
streamingllm	32.71	90.56	41.40	96.81	41.86	82.15	43.10	99.29	44.77	102.97

Table 3: PG-19 CPU results. PPL is lower better; throughput is higher better.

Method	0.2		0.4		0.5		0.6		0.8	
	PPL	tok/s	PPL	tok/s	PPL	tok/s	PPL	tok/s	PPL	tok/s
none	23.51	89.69	23.51	89.69	23.51	89.69	23.51	89.69	23.51	89.69
adakv	25.08	80.36	26.75	78.31	27.03	79.80	27.18	72.26	27.77	72.65
knorm	25.21	92.89	27.06	97.02	27.37	96.88	28.27	100.91	31.92	88.31
snapkv	23.52	91.41	23.87	98.44	23.94	100.01	23.95	97.55	24.93	98.65
streamingllm	23.32	94.09	23.57	94.96	23.99	94.22	23.91	99.72	24.34	103.22

3 Results

3.1 WikiText-2

Table 2 reports perplexity and decode throughput on WikiText-2. The dense baseline has PPL 32.30 and throughput 65.89 tokens/s. At low compression ratio 0.2, SnapKVPress and StreamingLLMPress nearly preserve baseline perplexity while improving throughput by about 40% and 37%, respectively. Higher compression usually improves speed but hurts PPL.

3.2 PG-19

Table 3 reports the same sweep on PG-19. The dense baseline has PPL 23.51 and throughput 89.69 tokens/s. StreamingLLMPress is the most robust method at low compression in this run: at ratio 0.2 it slightly improves PPL to 23.32 and increases throughput to 94.09 tokens/s. SnapKVPress also preserves quality well, staying below PPL 25 even at ratio 0.8.

4 Analysis

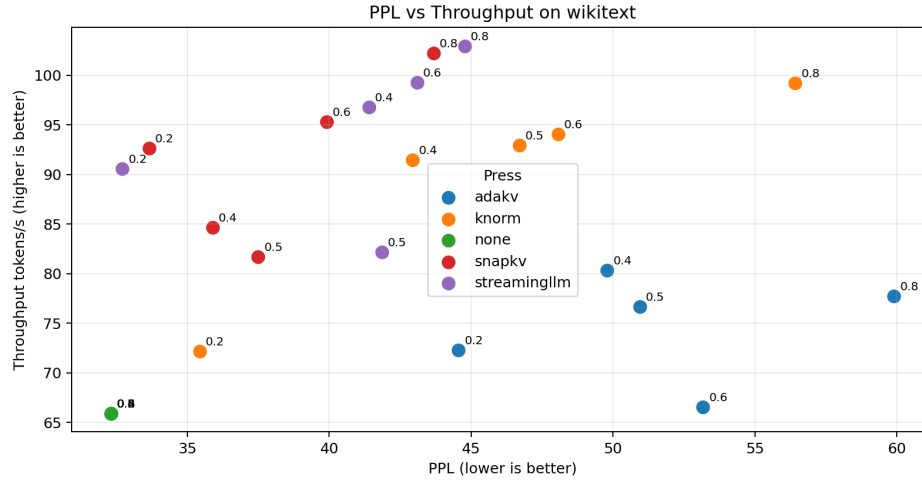
Compression improves decode throughput because each generated token attends to a shorter KV cache. Since timing starts after compression, the measured speedup mostly comes from reduced attention work and memory traffic. PPL rises when pruned tokens contain information needed for the target continuation.

SnapKVPress performs well because recent-window attention is a direct importance signal for the current context. StreamingLLMPress is also strong because sink tokens and recent tokens are often enough for stable local continuation. KnormPress is cheaper but less context-aware, so it loses more quality at high compression. AdaKVPress does not improve over SnapKV here; its head-wise budget flexibility adds complexity but does not give a better tradeoff for this small Pythia-70m CPU run.

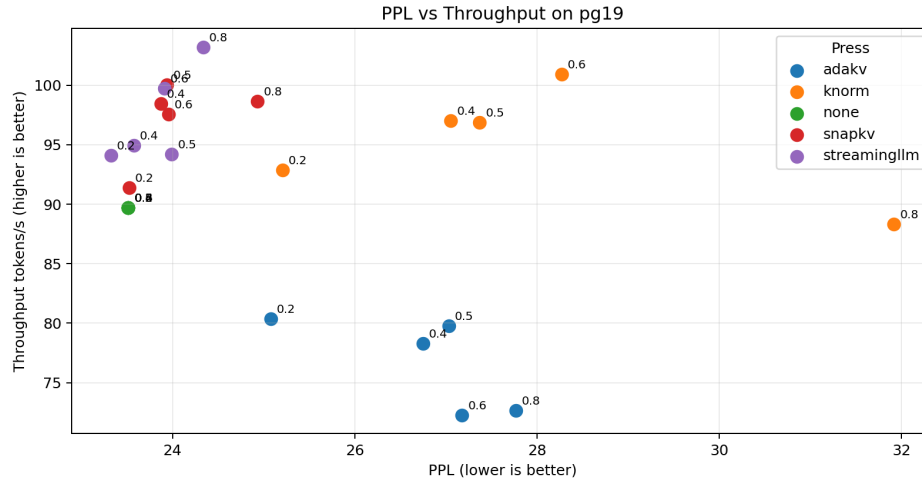
The best conservative WikiText-2 point is SnapKVPress at ratio 0.2: PPL rises from 32.30 to only 33.65, while throughput increases from 65.89 to 92.67 tokens/s. On PG-19, StreamingLLMPress at ratio 0.2 and SnapKVPress at ratios 0.4–0.6 give the cleanest quality–speed balance.

5 Conclusion

The CPU results show the expected tradeoff: cache compression improves post-compression decode throughput, but aggressive pruning can hurt perplexity. SnapKVPress is the most consistent quality-



model=EleutherAI/pythia-70m, device=cpu, context_tokens=1024, target_tokens=256, samples=3



model=EleutherAI/pythia-70m, device=cpu, context_tokens=1024, target_tokens=256, samples=1

Figure 1: Perplexity–throughput tradeoff on WikiText-2 and PG-19. Points closer to the lower-right are better.

44 preserving method, while StreamingLLMPress is a strong simple speed baseline. Future runs should
 45 use more PG-19 samples and repeat the sweep on CUDA.

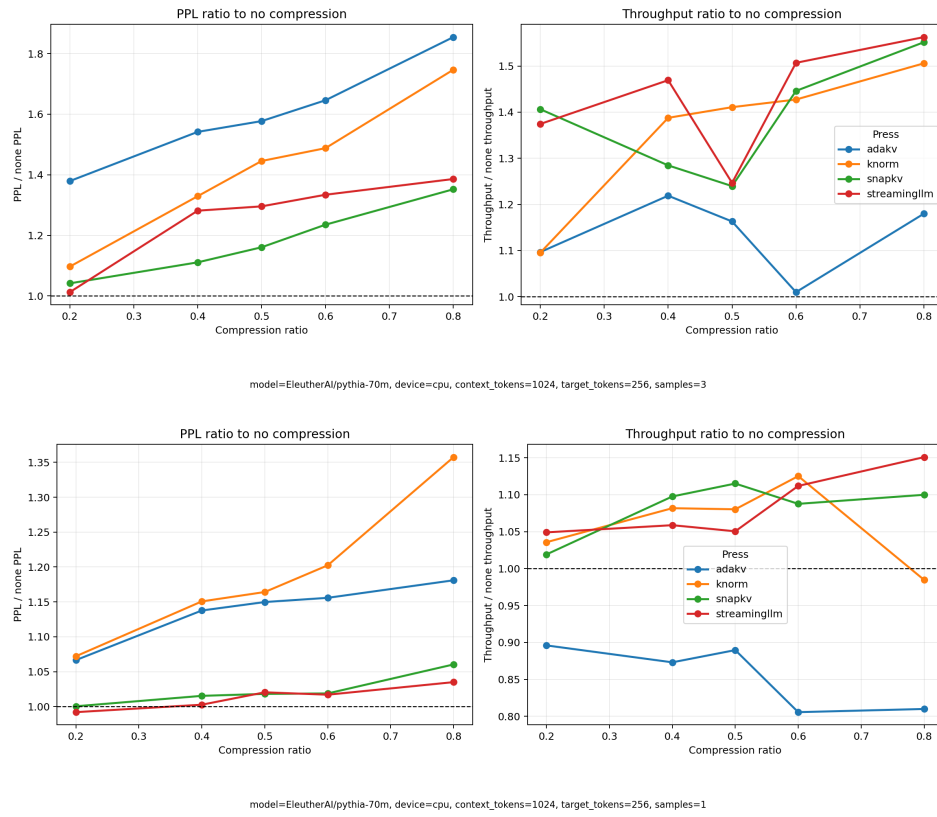


Figure 2: Relative PPL and throughput compared with the no-compression baseline at each compression ratio.